

Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: A

Instructor Name: Miss Fozia Khan

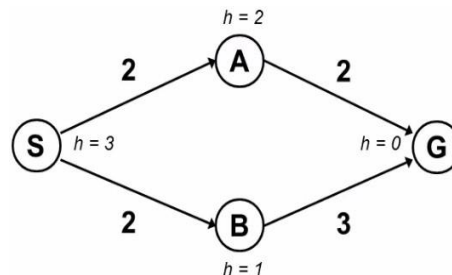
Course: Artificial Intelligence Lab (CS - 325L)

Program: (BS) - Computer Science

Lab #06

Activity- 1:

Implement A* search in the following state graph, find the shortest path between node S to Node G. Also solve it manually.



```
from collections import deque
import heapq
```

```
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start]) # Open list (queue)
    closed_list = [] # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft() # Dequeue the first node
        closed_list.append(node) # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start] # Open list (stack)
    closed_list = [] # Closed list (visited nodes)

    while open_list:
        node = open_list.pop() # Pop the last node (LIFO)
        closed_list.append(node) # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]): # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list # Return the lists if goal is not found

def a_star_search(graph, heuristic, start, goal):
    open_list = [] # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start)) # (f(n), node)
```

```

closed_list = [] # Closed list (visited nodes)
g_costs = {start: 0} # Cost from start to each node
came_from = {} # To reconstruct the path

while open_list:
    _, current = heapq.heappop(open_list) # Get the node with the lowest f(n)
    closed_list.append(current)

    # Check if the goal is reached
    if current == goal:
        path = reconstruct_path(came_from, current)
        return path, closed_list

    # Explore neighbors
    for neighbor, cost in graph[current]:
        tentative_g_cost = g_costs[current] + cost
        if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
            g_costs[neighbor] = tentative_g_cost
            f_cost = tentative_g_cost + heuristic[neighbor]
            heapq.heappush(open_list, (f_cost, neighbor))
            came_from[neighbor] = current

    return None, closed_list # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure based on the image
graph_bfs_dfs = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Graph structure with edge costs
graph_a_star = {
    'S': [('A', 2), ('B', 2)],
    'A': [('G', 2)],
    'B': [('G', 3)],
    'G': []
}

# Heuristic values for each node
heuristic = {
    'S': 3,
    'A': 2,
    'B': 1,

```

```

    'G': 0
}

# Initial and goal states for BFS and DFS
start_node_bfs_dfs = 'A'
goal_node_bfs_dfs = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)

# Display results
print("BFS Open List:", list(open_list))
print("BFS Closed List:", closed_list)

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)

# Initial and goal states for A* Search
start_node_a_star = 'S'
goal_node_a_star = 'G'

# Perform A* Search
path, closed_list = a_star_search(graph_a_star, heuristic, start_node_a_star, goal_node_a_star)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)

```

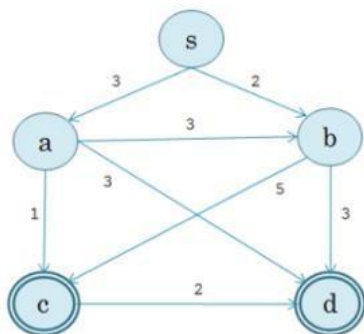
```

PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['S', 'A', 'G']
Closed List: ['S', 'B', 'A', 'G']

```

Activity- 2:

Implement A* search in the following state graph, find the shortest path between node 's' to node 'c' or 'd'. Also solve it manually.



h(s)	h(a)	h(b)	h(c)	h(d)
1	3	3	0	0

```
import heapq
```

```

def a_star_search(graph, heuristic, start, goals):
    open_list = [] # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start)) # (f(n), node)
    closed_list = [] # Closed list (visited nodes)
    g_costs = {start: 0} # Cost from start to each node

```

```

came_from = {} # To reconstruct the path

while open_list:
    _, current = heapq.heappop(open_list) # Get the node with the lowest f(n)
    closed_list.append(current)

    # Check if the goal is reached
    if current in goals:
        path = reconstruct_path(came_from, current)
        return path, closed_list

    # Explore neighbors
    for neighbor, cost in graph[current]:
        tentative_g_cost = g_costs[current] + cost
        if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
            g_costs[neighbor] = tentative_g_cost
            f_cost = tentative_g_cost + heuristic[neighbor]
            heapq.heappush(open_list, (f_cost, neighbor))
            came_from[neighbor] = current

return None, closed_list # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure with edge costs
graph = {
    's': [('a', 3), ('b', 2)],
    'a': [('c', 3), ('d', 5)],
    'b': [('a', 3), ('d', 3)],
    'c': [],
    'd': []
}

# Heuristic values for each node
heuristic = {
    's': 1,
    'a': 3,
    'b': 3,
    'c': 0,
    'd': 0
}

# Initial and goal states
start_node = 's'
goal_nodes = ['c', 'd'] # Multiple goals

# Perform A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_nodes)

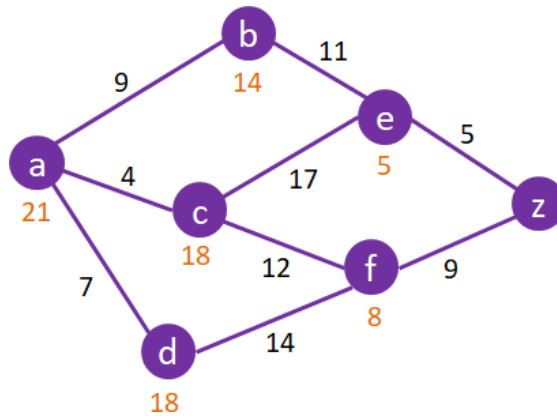
# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)

```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['s', 'b', 'd']
Closed List: ['s', 'b', 'd']
```

Activity- 3:

Implement A* search in the following state graph, find the shortest path between node 'a' to node 'z'. Also solve it manually.



```
import heapq
```

```
def a_star_search(graph, heuristic, start, goal):
    open_list = [] # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start)) # (f(n), node)
    closed_list = [] # Closed list (visited nodes)
    g_costs = {start: 0} # Cost from start to each node
    came_from = {} # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list) # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
```

```
# Graph structure with edge costs
graph = {
    'a': [('b', 9), ('c', 4), ('d', 7)],
    'b': [('e', 11)],
    'c': [('b', 17), ('f', 12)],
    'd': [('c', 14), ('f', 14)],
    'e': [('z', 5)],
    'f': [('e', 8), ('z', 9)],
    'z': []
}

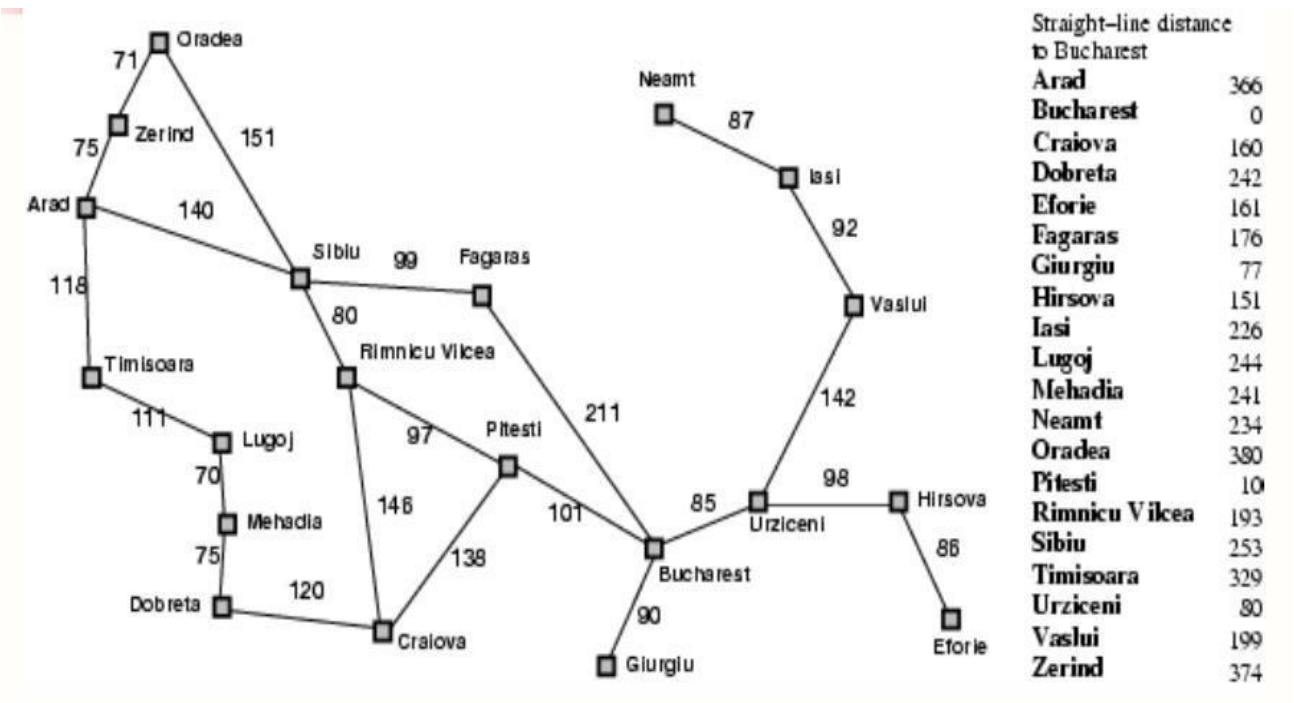
# Heuristic values for each node
heuristic = {
    'a': 21,
    'b': 14,
    'c': 18,
    'd': 18,
    'e': 5,
    'f': 8,
    'z': 0
}

# Initial and goal states
start_node = 'a'
goal_node = 'z'

# Perform A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

Activity- 4:
 Implement A* search in the following state graph, find the shortest path between node Arad to Node Bucharest. Also solve it manually.



```
import heapq
```

```
def a_star_search(graph, heuristic, start, goal):
    open_list = [] # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start)) # (f(n), node)
    closed_list = [] # Closed list (visited nodes)
    g_costs = {start: 0} # Cost from start to each node
    came_from = {} # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list) # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list # Return None if no path is found
```

```
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
```

```
# Graph structure with edge costs
```

```
graph = {
    'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101), ('Giurgiu', 90), ('Urziceni', 85)],
    'Giurgiu': [('Bucharest', 90)],
    'Urziceni': [('Bucharest', 85), ('Hirsova', 98), ('Vaslui', 142)],
    'Hirsova': [('Urziceni', 98), ('Eforie', 86)],
    'Eforie': [('Hirsova', 86)],
    'Vaslui': [('Urziceni', 142), ('Iasi', 92)],
    'Iasi': [('Vaslui', 92), ('Neamt', 87)],
```



```

    'Neamt': [('Iasi', 87)]
}

```

```

# Heuristic values for each node (straight-line distance to Bucharest)

```

```

heuristic = {
    'Arad': 366,
    'Bucharest': 0,
    'Craiova': 160,
    'Drobeta': 242,
    'Eforie': 161,
    'Fagaras': 176,
    'Giurgiu': 77,
    'Hirsova': 151,
    'Iasi': 226,
    'Lugoj': 244,
    'Mehadia': 241,
    'Neamt': 234,
    'Oradea': 380,
    'Pitesti': 100,
    'Rimnicu Vilcea': 193,
    'Sibiu': 253,
    'Timisoara': 329,
    'Urziceni': 80,
    'Vaslui': 199,
    'Zerind': 374
}

```

```

# Initial and goal states

```

```

start_node = 'Arad'

```

```

goal_node = 'Bucharest'

```

```

# Perform A* Search

```

```

path, closed_list = a_star_search(graph, heuristic, start_node, goal_node)

```

```

# Display results

```

```

print("Shortest Path:", path)

```

```

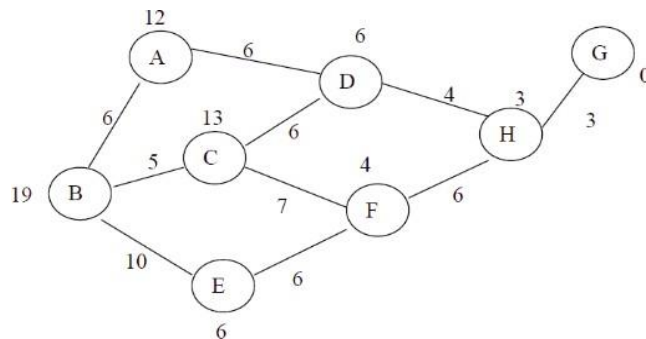
print("Closed List:", closed_list)

```

Activity- 5:

Implement Greedy Best First Search in the following state graph, find the shortest path between node 'A' to Node 'G'.

Also solve it manually.



```
import heapq
```

```
def greedy_best_first_search(graph, heuristic, start, goal):
    open_list = [] # Priority queue for open list
    heapq.heappush(open_list, (heuristic[start], start)) # (h(n), node)
    closed_list = [] # Closed list (visited nodes)
    came_from = {} # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list) # Get the node with the lowest h(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            if neighbor not in closed_list:
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))
                came_from[neighbor] = current

    return None, closed_list # Return None if no path is found
```

```
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
```

```
# Graph structure with edge costs
graph = {
    'A': [('B', 6), ('D', 6)],
    'B': [('A', 6), ('C', 5), ('E', 10)],
    'C': [('B', 5), ('D', 13), ('F', 7)],
    'D': [('A', 6), ('C', 13), ('H', 4)],
    'E': [('B', 10), ('F', 6)],
    'F': [('C', 7), ('E', 6), ('H', 4)],
    'H': [('D', 4), ('F', 6), ('G', 3)],
    'G': [('H', 3)]
}
```

```
# Heuristic values for each node
heuristic = {
    'A': 12,
    'B': 19,
    'C': 13,
    'D': 6,
    'E': 10,
    'F': 7,
    'H': 3,
    'G': 0
}
```

```
# Initial and goal states
```

```
start_node = 'A'
goal_node = 'G'
```

```
# Perform Greedy Best-First Search
```

```
path, closed_list = greedy_best_first_search(graph, heuristic, start_node, goal_node)
```

```
# Display results
```

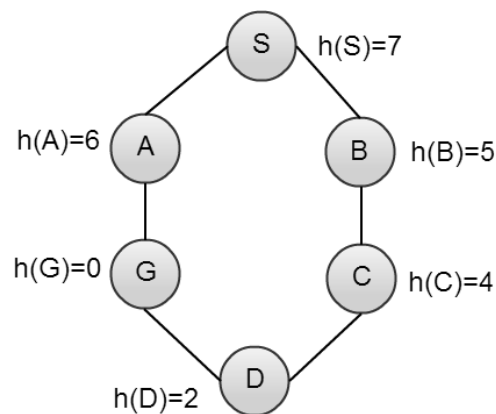
```
print("Shortest Path:", path)
```

```
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['A', 'D', 'H', 'G']
Closed List: ['A', 'D', 'H', 'G']
```

Activity- 6:

Implement Best First Search in the following state graph, find the shortest path between node 'S' to Node 'G'. Also solve it manually.



```
import heapq
```

```
def best_first_search(graph, heuristic, start, goal):
```

```
    open_list = [] # Priority queue for open list
```

```
    heapq.heappush(open_list, (heuristic[start], start)) # (h(n), node)
```

```
    closed_list = [] # Closed list (visited nodes)
```

```
    came_from = {} # To reconstruct the path
```

```
    while open_list:
```

```
        _, current = heapq.heappop(open_list) # Get the node with the lowest h(n)
```

```
        closed_list.append(current)
```

```
        # Check if the goal is reached
```

```
        if current == goal:
```

```
            path = reconstruct_path(came_from, current)
```

```
            return path, closed_list
```

```
        # Explore neighbors
```

```
        for neighbor, cost in graph[current]:
```

```
            if neighbor not in closed_list:
```

```
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))
```

```
                came_from[neighbor] = current
```

```
    return None, closed_list # Return None if no path is found
```

```
def reconstruct_path(came_from, current):
```

```
    path = [current]
```

```
while current in came_from:
    current = came_from[current]
    path.append(current)
path.reverse()
return path

# Graph structure with edge costs
graph = {
    'S': [('A', 1), ('B', 1)],
    'A': [('S', 1), ('G', 1)],
    'B': [('S', 1), ('C', 1)],
    'C': [('B', 1), ('D', 1)],
    'D': [('C', 1), ('G', 1)],
    'G': [('A', 1), ('D', 1)]
}

# Heuristic values for each node
heuristic = {
    'S': 7,
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 2,
    'G': 0
}

# Initial and goal states
start_node = 'S'
goal_node = 'G'

# Perform Best-First Search
path, closed_list = best_first_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['S', 'B', 'C', 'D', 'G']
Closed List: ['S', 'B', 'C', 'D', 'G']
```